

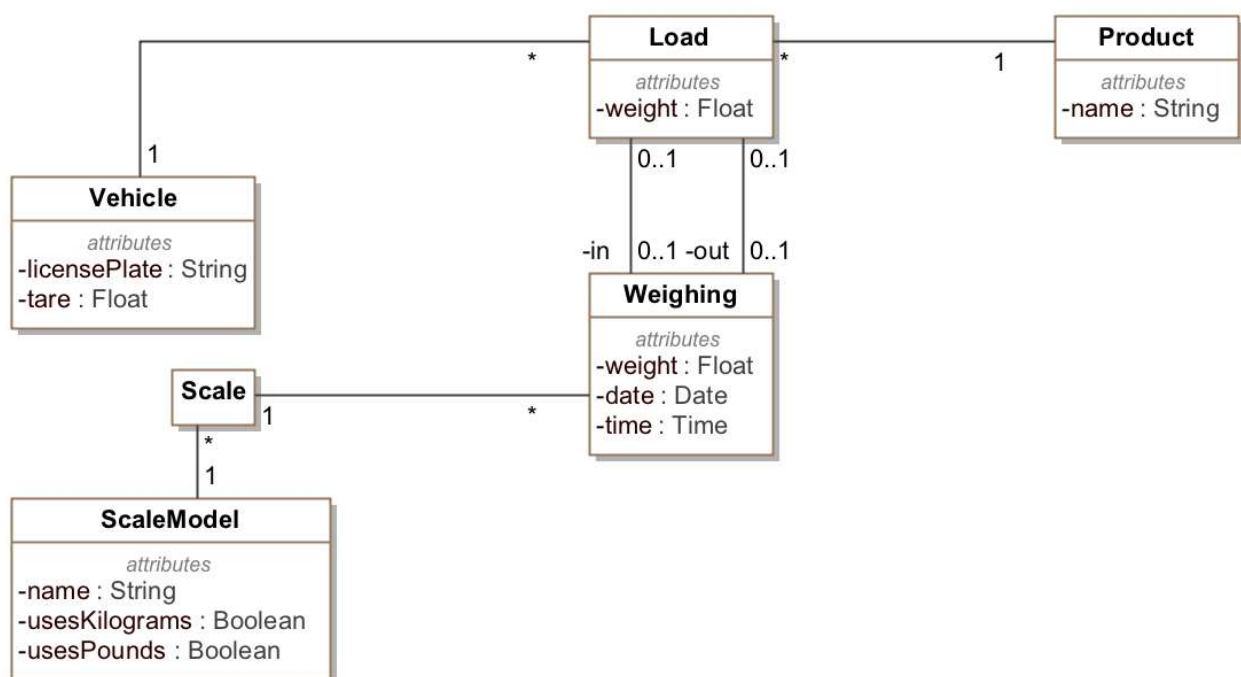
Análisis y Diseño con Patrones

Práctica 1: Patrones de análisis y arquitectónicos

Nos han contratado para desarrollar un software de control de mercancías que llegan al almacén del mercado central de una ciudad. En concreto, nos han pedido ayuda con la aplicación de control de pesaje de la báscula: Cuando un camión llega cargado de producto (por ejemplo, de trigo) y declara una carga concreta, se pesa el camión en llegar al mercado y al salir de manera que la diferencia entre los dos pesajes debe cuadrar con el peso declarado de la mercancía.

Por ejemplo, si un camión llega cargado con 100 kg de azúcar a granel, al salir debe pesar 100 kg menos de los que pesaba al entrar. De hecho, el pesaje de salida debería ser similar a la tara (peso en vacío) del camión

Los analistas del almacén nos han dado el siguiente diagrama de clases para que lo usemos como punto de partida:



Restricciones de integridad:

- El peso de un cargamento con pesaje de entrada y de salida debe ser igual a la diferencia entre los dos pesajes (con un margen de error del 5%).
- El peso de salida de un vehículo debe ser mayor o igual que su tara (que es el peso del vehículo sin personal de servicio, pasajeros ni carga y con la dotación completa de agua, combustible, etc.).
- Un modelo de báscula debe tener `usesKilograms` o `usesPounds` a true y sólo uno de los dos puede ser true.

Observaciones:

- El peso de un cargamento siempre está en kilogramos.

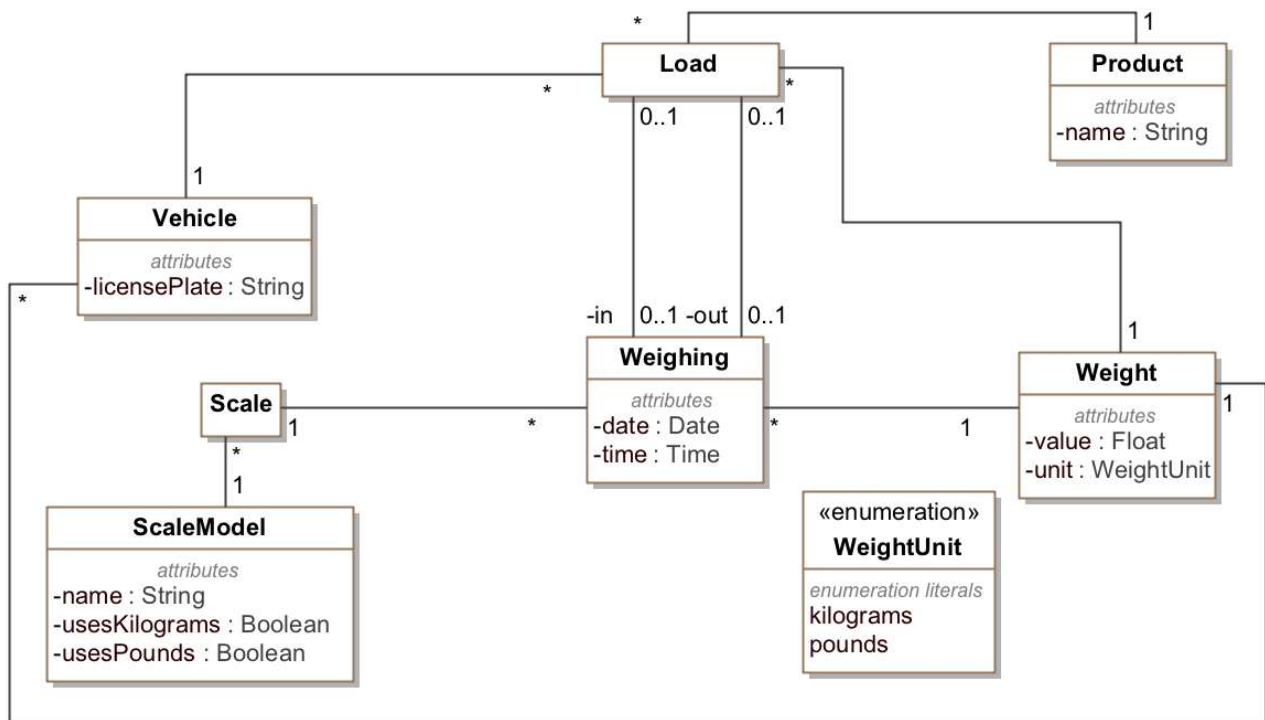
Ejercicio 1 (20%)

Como podemos ver en el diagrama de clases, los diferentes modelos de báscula pueden utilizar unidades diferentes. Esto debe tenerse en cuenta a la hora de calcular el peso ya que se han de hacer las conversiones adecuadas.

¿Cómo mejorarías la solución propuesta para tener en cuenta este hecho? Si aplicarías algún patrón, indica qué patrón has aplicado y razona la conveniencia de su aplicación.

Solución:

Aplicaría el patrón Cantidad para que los pesajes tuvieran en cuenta la unidad en que está cada pesaje:



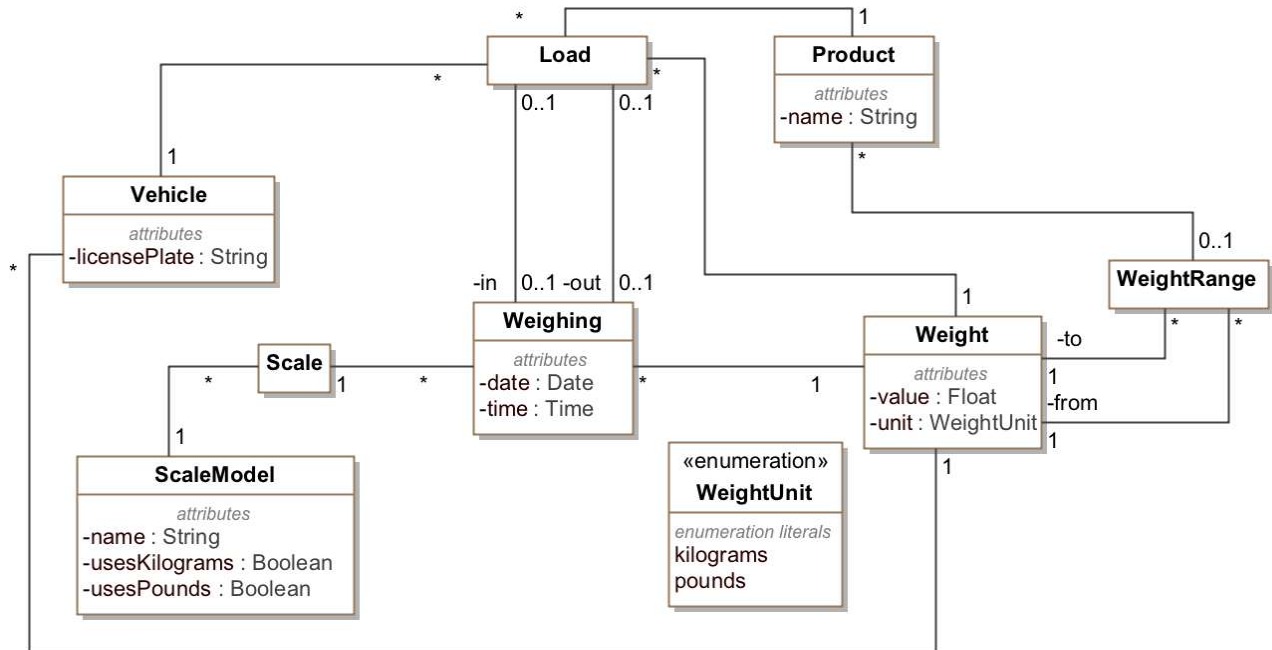
Ejercicio 2 (20%)

Debido a restricciones en las condiciones de almacenamiento (por ejemplo, que se requiera refrigeración) o de transporte (por ejemplo, porque no sale a cuenta guardar 300 gramos de azúcar en el almacén) algunos productos requieren un peso mínimo en cada carga y un peso máximo que deben tener las cargas de aquel producto que lleguen al almacén.

¿Cómo modificarías el diagrama estático del análisis para registrar esta información? En concreto, aplicarías algún patrón de los que hemos visto a este caso? En caso afirmativo, indica cuál. Justifica tu respuesta.

Solución:

Aplicaría el patrón Rango para representar el rango de valores válidos para cada tipo de producto:



Ejercicio 3 (20%)

Supongamos que existe una operación en la clase *Load* llamada *isConsistent* que analiza el pesaje de salida y lo compara con la tara del camión para decir si es consistente con la restricción de integridad o no. Esta operación está implementada así:

```

class Load {
    public boolean isConsistent () {
        if(out.isEmpty ()) {
            return true;
        }
        float factor = out.scale.scaleModel.usesKilograms ? 1: 0.4536
        float weightInKilos = out.weight * factor
    }
}

```

```

        return vehicle.tare <= weightInKilos
    }
}

```

¿Hay algún principio de diseño que se esté incumpliendo? En caso afirmativo, indica cuál es el principio de diseño, justifica por qué crees que se está incumpliendo y propón una solución alternativa. En caso negativo, justifica por qué no crees que se esté incumpliendo ningún principio de diseño.

Solución:

Se está incumpliendo el principio de "Bajo acoplamiento" ya que la clase Load está acoplada con Weighing, Scale y ScaleModel. Se podría implementar esta operación aprovechando que hemos aplicado el patrón Cantidad con mucho menos acoplamiento de la siguiente manera:

```

class Load {
    public boolean isConsistent () {
        if(out.isEmpty ()) {
            return true;
        } else {
            return vehicle.weight.isLessThan(out.weight)
        }
    }
}

```

En este caso, out.weight es de tipo Weight y, como vehicle.tare también es de tipo Weight, esta clase sabe hacer la comparación teniendo en cuenta la unidad sin problemas.

Ejercicio 4 (20%)

Supongamos que la clase *Scale* tiene estas operaciones:

```
public class Scale {
    public void drawWeight (screen: Screen) {
        //Pinta para la pantalla 'screen' el peso actual
    }
    public void storeCurrentWeight (file: File) {
        //Guarda a archivo el peso actual
    }
    public float getCurrentWeight () {
        //Devuelve el peso actual a partir de los datos del sensor
    }
    public boolean usesKilograms () {
        //Nos dice si una báscula utiliza kilogramos para el peso
    }
    public String getModelName () {
        //Devuelve el nombre del modelo de la báscula
    }
}
```

¿Crees que esta clase cumple todos los principios de diseño que hemos visto? En caso afirmativo, justifica porque crees que es así. En caso negativo, indica qué principios crees que no cumple y razona tu respuesta (no es necesario dar una solución alternativa que resuelva los problemas detectados).

Solución:

Esta clase incumple el principio de "Alta Cohesión" porque tiene operaciones con responsabilidades de diferentes niveles de abstracción como pintar cosas por pantalla, escribir a ficheros o implementar la lógica del dominio.

Ejercicio 5(10%)

Tenemos el siguiente caso de uso:

Caso de uso: Registrar pesaje

Resumen de la funcionalidad: Grabar un pesaje en el sistema

Flujo de eventos principal:

1. El usuario selecciona la báscula (puede haber más de una)
2. El usuario selecciona el vehículo de la lista de vehículos registrados en el sistema y el producto que lleva (de la lista de productos registrados en el sistema)
3. El sistema indica si el pesaje es de entrada o de salida (mirando si ya tiene un pesaje de entrada para este vehículo) y muestra el peso registrado por el sensor de la báscula
4. El usuario acepta el peso
5. El sistema registra el pesaje como pesaje de entrada o salida asociado al vehículo, la báscula y el producto indicados.

Flujo de eventos alternativo:

- 2a) El usuario da de alta un nuevo vehículo al sistema y volvemos al paso 2
- 2b) El usuario da de alta un nuevo producto y volvemos al paso 2
- 4a) El usuario modifica el valor del peso manualmente y volvemos al paso 5

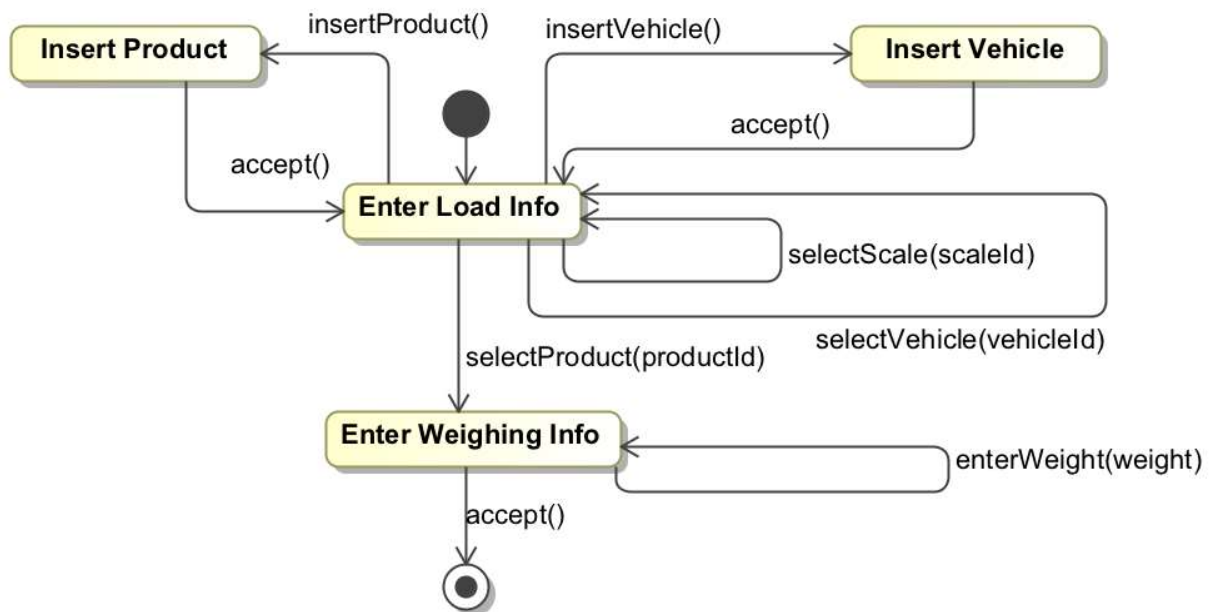
Haz el modelo de interfaz de usuario correspondiente a este caso de uso.

Fe de errores: Este párrafo que aparecía el ejercicio 6 hace referencia a este ejercicio (el 5):
Ten en cuenta que:

- Puede dibujar las pantallas a mano alzada o utilizar cualquier herramienta que desee
- Puede utilizar datos inventados a las pantallas si cree que así se entienden mejor
- Puede hacer los supuestos que considere oportunos siempre y cuando los documente claramente a su solución

Solución:

En primer lugar haremos el diagrama de estados para ver qué pantallas tenemos en el sistema:



La pantalla **EnterLoadInfo** nos debe permitir elegir la báscula, el vehículo y el producto. También nos debe permitir crear un nuevo vehículo o producto. Una vez elegido el producto iremos a la pantalla **EnterWeighingInfo**.

Enter Load Info

Scale

▼

Vehicle

▼

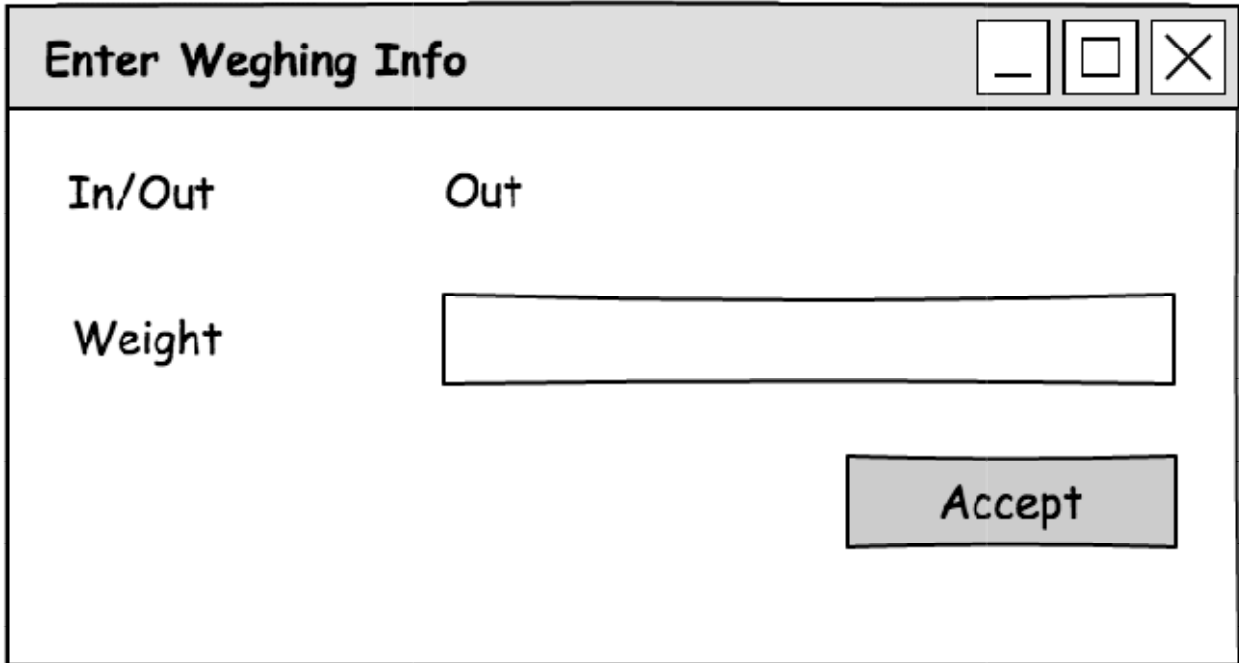
Add

Product

▼

Add

La pantalla EnterWeighingInfo nos debe mostrar si el pesaje es de entrada o de salida (esto lo calcula el sistema) así como el peso (que se debe poder modificar). También debe dar la opción de aceptar el pesaje.



Las pantallas InsertProduct y InsertVehicle no están desarrolladas en caso de uso así que podemos suponer que son muy sencillas y sólo nos tienen que mostrar un formulario para introducir los datos del vehículo (matrícula y tara) o del producto (nombre) con un botón de aceptar y un cancelar.

Insert Vehicle

License Plate

Tare (kg)

Accept

Insert Product

Name

Accept

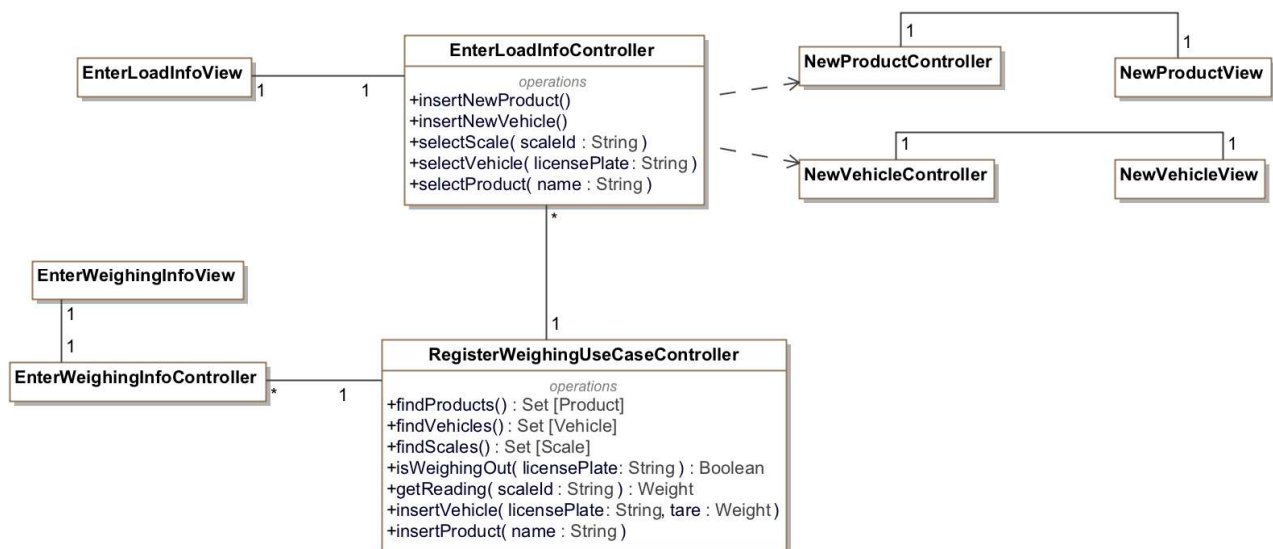
Ejercicio 6 (10%)

Hemos decidido que el sistema que queremos construir siga una arquitectura en capas (con una capa de presentación, una de dominio y una de servicios técnicos). Queremos definir la capa de presentación de nuestro sistema y para ello aplicamos el patrón de arquitectura Modelo, Vista y Controlador (MVC). Proponga un diagrama de clases donde se aplique el MVC para el caso de uso Registrar pesaje.

Para cada clase define la firma de las operaciones correspondientes al tratamiento en la capa de presentación cuando el usuario inicia el caso de uso hasta que el sistema registra el pesaje.

Solución:

Para solucionar este ejercicio necesitaremos poder identificar las balanzas por algún tipo de id que no está en el ejercicio inicial ya que hasta ahora no lo habíamos necesitado.



Ten en cuenta que:

- Puedes dibujar las pantallas a mano alzada o usar cualquier herramienta que desee
- Puedes utilizar datos inventados en las pantallas si crees que así se entienden mejor

Puedes hacer las suposiciones que consideres oportunas siempre y cuando las documentes claramente en la solución